

Estándar de codificación para C# y .NET

Índice

Introducción	2
Propósito.....	3
Idioma	4
Reglas de nombramiento	5
Variables.....	5
Campos y eventos.....	5
Parámetros.....	6
Métodos	6
Interfaces	7
Clases.....	7
Abreviaturas válidas	9
Prefijos	9
Estilos de código	11
Indentación	11
Líneas	11
Espacios en blanco.....	12
Uso de llaves	12
Estructuras de control.....	13
Namespaces	18
Nombramiento	18
Comentarios.....	19
Comentarios de línea única.....	19
Comentarios de bloque.....	20
Comentarios de API pública	21
Manejo de Excepciones.....	22
Manejador de log (bitácora)	22
Identificación de tipos de error	23
Bloques Try-Catch	23
Pruebas	25
Nombrado de pruebas	25
Seguridad	26
Declaración de IA	27
Bibliografía.....	28

Tecnologías para la Construcción de Software

Docente: Juan Carlos Pérez Arriaga

Estudiantes:

Leonardio Daniel Ortega Teoba

Gabriel Antonio González López

Universidad Veracruzana, Facultad de Estadística e Informática, Ingeniería de Software

Introducción

Este estándar está dirigido para estudiantes de Tecnologías para la Construcción de Software en la carrera de Ingeniería de Software, se recomienda usarlo para el desarrollo de los proyectos en clase y productos de software, con el objetivo de establecer una base para la construcción y lectura del código, promoviendo buenas prácticas de programación (convenciones del lenguaje brindadas por Microsoft), su legibilidad, orden, espaciado y nombramiento, accesibilidad a su mantenimiento para aquel que busque modificarlo y probarlo para verificar su seguridad y funcionamiento en el desarrollo de este producto de software.

Propósito

Este estándar tiene como propósito brindar especificaciones que ayuden a desarrollar un código seguro, mantenible y legible con C#. Así como brindar un seguimiento de parámetros definidos para contar con una guía clara y estructurada para el desarrollo de software apegada completamente a las siguientes pautas que hemos definido.

Se entiende como código seguro a aquel que no sea susceptible a inyecciones de código, acceso a datos sensibles no cifrados o sin una verificación de credenciales previa.

Idioma

Para el nombramiento del código se usará el idioma inglés. Se tomó esta decisión para motivar a los estudiantes a desarrollar y mejorar su dominio del idioma, así como adaptarse al ambiente laboral general del mundo.

En cuanto al lenguaje de la GUI (Graphic User Interface) puede ser en cualquier idioma, los proyectos pueden incluir varios perfiles de idiomas.

Reglas de nombramiento

Los nombres deben ser descriptivos, independientemente de si se está nombrando un método, variable o clase. Solo bajo algunos casos se podrán usar abreviaciones cuyo significado se podrá encontrar al final de la sección.

Variables

Para las variables se usará **camelCase** y un nombre descriptivo.

Buen uso

```
int firstNumber;  
string clientOrder;  
bool isPrime = false;
```

Mal uso

```
int AccountTotal = 0;  
string Message = "hello world";  
int MAX_RND = 2000;
```

Campos y eventos

Los campos y eventos de las clases deberán ser escritas con **PascalCase** (sin importar si son públicos, privados, internos, etc).

Buen uso

```
private int MaxOfAccount;  
public static Random RandomInstance;  
private event Action OnMessageNotificationReceive;
```

Mal uso

```
private int number_of_orders;  
public static Queue csmrQ;  
private readonly Message;
```

Las variables constantes también deberán usar **PascalCase**.

Buen uso

```
private const double Pi = 3.1459;
```

```
public const int MaximumTry = 3;
```

Mal uso

```
private const int a = 10;  
public const double EULER = 2.71828;
```

Parámetros

Para los parámetros se deberá usar camelCase junto con nombres que sean descriptivos y que reflejen un propósito y sean consistentes, se debe evitar el uso de abreviaciones.

Buen uso

```
public void AddMountToClientCash(ref decimal clientCash, decimal mount)  
{  
    clientCash += mount;  
}
```

Mal uso

```
public int UpdateValue(ref decimal x, decimal y)  
{  
    x += y;  
}
```

Métodos

Los métodos se deben nombrar según su **funcionalidad** para facilitar su lectura y comprensión, se usará *PascalCase*.

Buen uso

```
public void CalculateSummatory(int aNumber, int bNumber)  
{  
    return aNumber + bNumber;  
}
```

Mal uso

```
public int calculatesum(int a, int b)  
{  
    return a + b;  
}
```



```
public void P()
{
    Console.WriteLine("HelloWorld");
}
```

Interfaces

Las interfaces deben comenzar con la letra I mayúscula (de Interface) seguido por el nombre de la interfaz escrito en *PascalCase*. El nombre debe ser descriptivo y representar una capacidad o contrato que implementan las clases.

Cuando sean comportamientos se deben usar adjetivos o participios que terminen en "able" o "ible".

Buen uso

```
public interface IPrinter
{
    void Print();
}
```

Mal uso

```
public interface Client
{
    void DoSomething();
}

public interface IL()
{
    void Login();
}
```

Clases

Las clases deberán ser nombradas con un sustantivo descriptivo y escritas con *PascalCase*.

Buen uso

```
public class Client
{
    private string Name { get; set; }
}
```

Mal uso

```
public class myClass
{
    public string race;
    public string name;
}
```

Métodos get y set

Las propiedades deben ser escritas en PascalCase y deben ser sustantivos que describen el valor.

Se utiliza get para obtener valores y set para asignar respectivamente, siempre se escriben en minúsculas debido a que forman parte del lenguaje.

Se pueden crear propiedades personalizadas para agregar lógica adicional al leer o asignar un valor.

Se utilizan las propiedades para:

- Validar datos antes de asignarlos.
- Aplicar reglas de negocio.
- Transformar información antes de devolverla.

No se debe ocupar verbos como GetId o SetAge.

Buen uso de propiedades automáticas

```
public class Client
{
    public string Name { get; set; }
}
```

Buen uso de propiedades personalizadas

```
public class Client
{
    private int PrivateAge;
    public int Age
    {
        get { return age; }
        set
        {
            if (value < 0)
            {
                throw new ArgumentException("Age cant' be negative");
            }
        }
    }
}
```

```

        PrivateAge = value;
    }
}

```

Mal uso de propiedades automáticas

```

public class Client
{
    public string getName {get; set; }
}

```

Abreviaturas válidas

Abreviatura	Significado
DAO	Data Access Object
DB	DataBase
Rand	Random
Max	Maximum
Min	Minimum

Prefijos

Específicamente en la capa gráfica se permitirán el uso de prefijos, a continuación se nombrarán los prefijos utilizados y sus significados, no deberán usarse otros que no estén en esta lista. Se deja a consideración del desarrollador si usarlos o no.

Prefijo	Significado
Btn	Button
Lbl	Label
TxtBk	TextBlock
TxtBx	TextBox
PwdBx	PasswordBox
ChkBx	CheckBox
RBtn	RadioButton
CbBox	ComboBox
LstBox	ListBox
Sli	Slider
PBar	ProgressBar

Img	Image
MdElem	MediaElement
Lay	Layout
Stack	StackPanel

Buen uso

```
<StackPanel x:Name="InteractionStack" Orientation="Vertical">
  <TextBlock x:Name="TxtbkTitle2" Text = "Super App" />
  <Slider x:Name="sliUserAge" Minimum="1" Maximum="120" />
  <Button x:Name="BtnPlay" Content="Play" />
  <Button x:Name="BtnQuit" Content="Quit" />
</StackPanel>
```

Mal uso

```
<StackPanel x:Name="istack" Orientation="Vertical">
  <Label x:Name="lbAge" />
  <Slider x:Name="SAge" Minimum="1" Maximum="110" />
  <Button x:Name="ButtQuit" Content="Quit" />
</StackPanel>
```

Estilos de código

Indentación

Para los espacios de indentación deberán usarse espacios en blanco como media, cada nivel equivale a 4 espacios. Un "tab" será igual a 4 espacios en blanco.

Buen uso

```
public class Person
{
    private string PrivateName;
    public string Name { get; set; }

    public void PrintName(int repetitions)
    {
        for (int i = 0; i < repetitions; i++)
        {
            Console.WriteLine(name);
        }
    }
}
```

Mal uso

```
public class Person
{
    private string PrivateName;
    public string Name { get; set; }
}
```

Líneas

Cada línea debe tener por mucho 120 caracteres, en caso de que se exceda esa cantidad se debe continuar en la siguiente línea cuando se encuentre alguno de los siguientes caracteres:

- Operadores matemáticos (+, -, *, /).
- Paréntesis (), cuando se abren o cierran.
- Comas (,), especialmente en listas de argumentos.
- Llaves ({}), si es el inicio o fin de un bloque de código.
- Corchetes ([]), cuando se declaran o acceden a arreglos.

Buen uso

```
string textType = "Very long text";
string myVeryLongText = "This is a " + textType +
    " I came up with");
```

Mal uso

```
string myVeryLongText = "This is a very long text that should be split but it's not,
makint it hard to read and maintain in any code editor without horizontal scrolling."
```

Espacios en blanco

Se usará un espacio en blanco cada vez que escribamos una palabra (el nombre de una variable cuenta como una palabra) o caracter:

Buen uso

```
int totalSummatory = firstNumber + secondNumber;
Console.WriteLine("Calculation result: " + totalSummatory);

for (int currentIteration = 0; currentIteration < 10; currentIteration++)
{
    Console.WriteLine("Iteration: " + currentIteration);
}
```

Mal uso

```
int sum=a+b;
Console.WriteLine("Result: "+sum);
for(int i=0;i<10;i++)
{
    Console.WriteLine(i);
}
```

Uso de llaves

Las llaves de apertura de clases, interfaces y métodos deben estar una línea después de la línea en la que se hizo la declaración. La llave de cierre debe estar en una nueva línea sola e indentada para coincidir con el comienzo de la declaración.

Los bloques de estructuras de control (while, for, foreach, switch) deben estar encerrados con llaves, aunque solo contengan una línea, a excepción de los if.

Buen uso

```
public class Client() { public void Greet() { Console.WriteLine("Hello!"); } }
```

Mal uso

```
public class Client() { public void Greet() { Console.WriteLine("Hello!"); } }
```

Estructuras de control

if - else

Debe haber un espacio entre **if** y la condición, se deben utilizar llaves si el resultado es de más de una línea.

El **else** debe ir en la línea de abajo de la llave de cierre del **if**.

Buen uso

```
public void CheckAge(int studentAge)
{
    if (studentAge >= 18)
    {
        Console.WriteLine("The student is an adult.");
    }
    else
    {
        Console.WriteLine("The student is underage.");
    }
}
```

Mal uso

```
public void CheckAge(int studentAge)
{
    if(studentAge >= 18) {
        Console.WriteLine("The student is an adult.");
    } else { Console.WriteLine("The student is underage."); }
}
```

If de una línea

Cuando el procedimiento dentro de un **if** es de solo **una** línea y no es seguido por un **else** se pueden omitir los corchetes, esto con el fin de mejorar la legibilidad.

Buen uso

```
public void CheckAge(int studentAge)
{
    if (studentAge >= 18)
        Console.WriteLine("The student is an adult.");
}
```

```
}
```

Mal uso

```
public void CheckAge(int studentAge)
{
    if (studentAge >= 18) Console.WriteLine("The student is an adult.");
    else
        Console.WriteLine("The student is underage.");
}
```

Else If

La estructura tiene que comenzar en la línea debajo de la última llave del bloque **if**. La estructura **else if** tiene que ir seguida por un espacio en blanco antes de la condición y la llave de inicio.

Buen uso

```
public void CheckGrade(int grade)
{
    if (grade >= 8)
    {
        Console.WriteLine("Excellent!");
    }
    else if (grade >= 6)
    {
        Console.WriteLine("Good!");
    }
    else
    {
        Console.WriteLine("You can do it better!");
    }
}
```

Mal uso

```
public void CheckGrade(int grade)
{
    if(grade >= 8)
    {
        Console.WriteLine("Excellent!");
    }
    else
    if(grade >= 6)
    {
        Console.WriteLine("Good!");
    }
}
```



```
    else
    {
        Console.WriteLine("You can do it better!");
    }
}
```

Switch

Debe declararse en una nueva línea y debe tener un espacio antes de la condición. Cada uno de los **case** deben ser indentados un nivel.

Buen uso

```
public void Menu(int option)
{
    switch (option)
    {
        case 1:
            Console.WriteLine("You select the option 1");
            break;
        default:
            Console.WriteLine("Select a valid option");
            break;
    }
}
```

Mal uso

```
public void Menu(int option)
{
    switch(option)
    {
        case 1:
            Console.WriteLine("You select the option 1");
            break;
        default:
            Console.WriteLine("Select a valid option");
            break;
    }
}
```

While

Debe existir un espacio entre la palabra **while** y la condición. Siempre se deben utilizar llaves {}, incluso cuando el bloque contiene únicamente una línea.

Buen uso

```
while (numberA >= 0)
{
    Console.WriteLine(numberA);
}
```

Mal uso

```
while(true)
    Console.WriteLine("Executting...");
```

Do-While

En la línea siguiente a la palabra **do** debe ir la llave de inicio, la palabra **while** debe ir en la misma línea de la llave de cierre, separado por un espacio de la condición.

Buen uso

```
int numberA = 0;
do
{
    Console.WriteLine(numberA);
    i++;
} while (numberA <= 5);
```

Mal uso

```
string input;
do
{
    input = Console.ReadLine();
} while (!CheckInput(input));
```

For

Debe haber un espacio entre la palabra **for** y la condición, la declaración dentro de los paréntesis debe estar separada por **;** junto con un espacio después de cada uno, la llave de inicio deberá ir en la línea siguiente a la declaración del **for**.

Buen uso

```
int evenCounter = 0;
for (int currentNumber = 1; currentNumber <= 20; currentNumber++)
{
    if (currentNumber % 2 == 0)
    {
```

```
        evenCounter++;  
    }  
}  
Console.WriteLine("Even numbers:" + evenCounter);
```

Mal uso

```
int sum = 0;  
for(int currentNumber = 0;currentNumber<10;currentNumber++)  
{  
    sum += currentNumber;  
}  
Console.WriteLine("Sum:" + sum);
```

Namespaces

Para los namespaces, los cuales son necesarios para crear claridad para el programador para que cuando se usen sepa perfectamente qué es lo que contiene. Deberán tener la siguiente estructura:

```
<Compañía>.(<Producto>|Tecnología>)[.<Feature>][.<Subnamespace>]
```

Nombramiento

Se deben usar prefijos para nombrar los namespaces, estos se recomienda que sean las compañías.

Se debe evitar usar nombres dependientes de la versión para el segundo nivel (Producto|Tecnología). El uso *PascalCase* para escribirlos y separar componentes del namespace con puntos es necesario.

Se pueden usar nombres en plural cuando sea apropiado, pero no se debe usar el mismo nombre para un namespace y tipo en ese namespace. Tampoco deberán usarse nombres genéricos como Element, Node, Log y Message.

Buen uso

```
FEI.Eminus.ActivityAssignment  
System.Threading.Tasks  
Forbidden.Test
```

Mal uso

```
fei.ProPractices5_0StudentManagement  
Calculator.sqr
```

Comentarios

El uso de comentarios solo será permitido cuando la implementación que se dio es fuera de lo normal o requiere una explicación más avanzada y para anotar banderas de desarrollo.

Comentarios de línea única

Los comentarios de línea única deben estar rodeados de una línea en blanco, se usarán para expresar:

- **TO-DO:** Funcionalidad que debe codificarse
- **FIX:** Se debe arreglar alguna parte de ese código o funcionalidad
- **TEST:** Se necesita probar esa funcionalidad
- **REFACTOR:** Se debe refactorizar esa funcionalidad

El mensaje debe estar entre dobles diagonales, un espacio y tres asteriscos:

```
// *** TO-DO *** //
```

Buen uso

```
// *** FIX *** //

public void FibonacciSequence()
{
}

// *** TO-DO *** //

public void EulerSequence()
{
}

// *** REFACTOR *** //

public async Task GetAllStudents()
{
}
```

Donde *GetAllStudents()* es un método de más de 30 líneas, viola el principio de responsabilidad única o tiene una complejidad ciclomática de más de 10, por poner algunos ejemplos.

Mal uso

```
// This method has x functionality
```

```
public void SuperCalculator()
{
}

public void foo(List<T> l)
{
    // iterate over list
    foreach (var something in l)
    {
        // process something
    }
}
```

Comentarios de bloque

Los comentarios de bloque se pueden ocupar (no son recomendados) únicamente para explicar una implementación extraña en algún método ya probado, por ejemplo:

- Implementaciones que no sean entendibles en una primera lectura.
- Soluciones que no siguen un patrón ya existente para ese problema. — Se debe explicar por qué no se usó la solución ya existente y probada.

En general, los comentarios jamás deben explicar qué hace el código, sino el por qué se escribió así (en caso de que requiera explicación).

Para comentar en bloque se usarán los comentarios multi línea (`/*, */`), se abrirá el comentario y se comenzará a escribir el comentario en la línea siguiente y se cerrará el comentario una línea después de que termine el mensaje.

Buen uso

```
/*
ShuffleCards is built this way because...
*/

public void ShuffleCards(List<Card> cards)
{
}
```

Mal uso

```
/* This method doesn't work well
*/

/*
This method is for...
*/
```

En el segundo ejemplo del mal uso es incorrecto porque la funcionalidad de un método debe poderse deducir por su nombre, por lo que un comentario no debe explicar el qué hace.

Comentarios de API pública

Conocidos como *Comentarios de Documentación*, se escribe con tres diagonales (`///`), es recomendable que incluya un summary mediante una etiqueta XML (`<summary></summary>`) donde se escribe un resumen del método y las etiquetas que correspondan, las más comunes son:

- `<param name="...">` Explica cada parámetro.
- `<returns>` Describe qué devuelve la función.
- `<remarks>` Agrega notas adicionales (a mayor detalle).
- `<example>` Muestra un ejemplo de uso.
- `<exception>` Describe la excepcion que la API puede tirar y bajo qué circunstancias.

Ejemplo

```
/// <summary>
/// Represents a basic calculator
/// </summary>
public class Calculator
{
    /// <summary>
    /// Subtraction of two numbers
    /// </summary>
    /// <param name="firstNumber"> The number we are subtracting of</param>
    /// <param name="secondNumber"> The number we are subtracting to firstNumber
</param>
    /// <returns> The remainder of firstNumber - secondNumber </returns>
    /// <example>
    /// Example of use:
    /// <code>
    /// var calculator = new Calculator();
    /// int result = calculator.Substraction(10, 3);
    /// </code>
    /// </example>

    public int Subtraction(int firstNumber, int secondNumber)
    {
        return firstNumber - secondNumber;
    }
}
```

Manejo de Excepciones

Las excepciones se deben propagar cuando los métodos donde puede ocurrir la excepción se llamarán desde otro método de otra clase, y en la case donde se llaman es donde se capturarán las excepciones lanzadas. Un ejemplo de esto es un DAO que puede lanzar un `SQLException`, dentro del DAO se propagan mientras que capturamos las excepciones cuando los mandamos a llamar.

En el caso de que un cliente llame a un método de un servidor el servidor registra en su log la excepción y le envía al cliente un *`FaultException`* con la información que le interesa.

Todos los errores se deben registrar en la bitácora (logger). En caso de que un error cause el cierre total del programa se debe informar al usuario que hubo un error inesperado y que se cerrará la aplicación, en casos como estos es de suma importancia que se registre en la bitácora. Se debe intentar perder la menor cantidad de información posible.

Siempre que ocurra un error se debe informar a los interesados, por ejemplo, si no se pudo escribir alguna actualización solicitada por el usuario se le deberá avisar mediante un mensaje en la pantalla, si hubo algún error de compilación o vulnerabilidad que no afecte directamente a la UX se deberá registrar en el logger e informar al desarrollador.

Las excepciones de tipo `Exception` se usarán como último recurso en el nivel de cacheo, es decir, primero se atraparán las excepciones específicas y por último la de tipo `Exception` en caso de ser necesario. Este manejo debe estar muy bien vigilado y justificado porque puede resultar en un antipatrón.

Manejador de log (bitácora)

Se debe usar **log4Net** para darle seguimiento a los errores y excepciones, se deberá mostrar el log en consola y en un archivo .log.

En el mensaje del log se debe indicar en qué clase y método se causó la excepción. De la siguiente manera: "Clase.Método", ya después irá la excepción.

Buen uso

```
catch (SQLException ex)
{
    logger.error("Trainer.CatchPokemon", ex);
}

catch (DivideByZeroException ex)
{
    logger.fatal("Calculator.Divide", ex);
}
```

Mal uso


```
catch (SQLException e)
{
    logger.error("Error reading something", e);
}

catch (EntityException e)
{
    logger.error(e);
}
```

Identificación de tipos de error

Fatales

Se consideran fatales las excepciones de las que la aplicación no podrá recuperarse y deberá cerrarse, por ejemplo:

- `OutOfMemoryException`
- `StackOverflowException`
- `FileNotFoundException` (de archivos críticos de la aplicación)

Errores

Se consideran errores las excepciones de las cuales sí se puede recuperar la aplicación, por ejemplo:

- `IOException`
- `NullPointerException`
- `EntityException`

Advertencias

Se consideran advertencias las excepciones que sean momentáneas, es decir, que por alguna razón extraordinaria sucedieron y si se vuelve a ejecutar el proceso no deberían volver a pasar. También entran en esta categoría las que no afecten el rendimiento y/o funcionamiento de la aplicación. Ejemplos:

- `TimeoutException`
- `ResourceNotFoundException`
- `FileNotFoundException` (En el caso de archivos que no afectan el funcionamiento del sistema, como fotos de perfil)

Bloques Try-Catch

Dentro del **try**, de preferencia, solo deberá estar el código que pueda lanzar una excepción, y la variable que almacena la excepción atrapada deberá llevar el nombre *ex*.

Las llaves deberán estar abajo de la declaración del **try** y **catch**.

Se evitará el uso de atrapar *Exception* genérica y se recomienda el uso de excepciones específicas.

Buen uso

```
try
{
    int[] numbers = {1, 2, 3};
    Console.WriteLine(numbers[5]);
}
catch (IndexOutOfRangeException ex)
{
    logger.error("NumberArray.PrintNumber" + ex);
}
```

Nota: Este fragmento siempre lanzará excepción, solo es un ejemplo.

Mal uso

```
try {
    int[] numbers = {1, 2, 3};
    Console.WriteLine(numbers[5]);
} catch (Exception e) {
    Console.WriteLine(e.Message);
}
```

Pruebas

Las pruebas deberán abarcar algunos de los siguientes casos si es que aplican:

1. Caso ideal
2. Entradas límite
3. Entradas inválidas
4. Comportamiento con datos duplicados
5. Efectos colaterales
6. Excepciones
7. Comprobación de retorno

Solo deberán contar con una aserción por prueba.

Nombrado de pruebas

El nombre de una prueba consta de tres partes, la primera es el prefijo Test, seguido por el método (o comportamiento) que se probará y la última parte será el camino que se está calificando (exitoso, entradas mínimas, entradas vacías, etc.).

Buen uso

```
public void TestLoginSuccessful()  
public void TestDatabaseConnectionWrongPassword()  
public void TestPlayersSyncInvalidToken()
```

Mal uso

```
public void RegisterTaskTest()  
public void CalculateSalaryWrong()  
public void TestConnectToDatabaseShouldReturnTrue()
```

Seguridad

Desarrollar un sistema seguro es de gran importancia, especialmente si será un sistema con conexión en red, por lo que mencionamos aquí las principales pautas a seguir para poder mantener la información íntegra y privada.

- Las contraseñas de los usuarios deberán estar hasheadas.
- Las credenciales del sistema para ingresar a bases de datos o algún otro sistema deberán guardarse en otro archivo de configuración al que se accederá desde el código.
 - Una opción es usar variables de entorno.
- **Jamás debe haber credenciales en el código.**
- El código deberá estar libre de smells y no deberá ser susceptible a inyecciones, esto se puede revisar con alguna herramienta de revisión de código como SonarQube.
- Las contraseñas deberán tener una longitud mínima de 8 caracteres y al menos un caracter de los siguientes:
 - Mayúsculas
 - Minúsculas
 - Número

Declaración de IA

Se declara el uso de IA para verificar la gramática del estándar, generación de ejemplos y comprensión de algunos temas.

Bibliografía

BillWagner. (n.d.-a). Identifier names - rules and conventions - C#. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>

BillWagner. (n.d.-b). .NET Coding Conventions - C#. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Chidera, I. (2023, January 13). C# Coding Best Practices – Coding Conventions with Examples. freeCodeCamp.org. <https://www.freecodecamp.org/news/coding-best-practices-in-c-sharp/>

De Tinchicus, V. T. L. E. (2020, October 27). C# / Getter and setter. El Blog De Tinchicus. <https://tinchicus.com/2020/10/27/c-getter-and-setter/>

BillWagner. (n.d.). Documentation comments - C# language specification. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/documentation-comments>

KrzysztofCwalina. (n.d.). Names of Namespaces - Framework design Guidelines. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/names-of-namespaces>

BillWagner. (n.d.-b). Instrucciones de control de excepciones: throw y try, catch, finally - C# reference. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/statements/exception-handling-statements>